

Cómo consumir un JSON desde Dart en Flutter

Con el catálogo CIE-11 de enfermedades laborales

Desde el archivo local hasta una lista en pantalla,

pasando por modelos, assets y peticiones HTTP.

01

Assets

Cargar el JSON empaquetado

02

Modelos

Mapear campos a clases Dart

03

UI

Mostrar datos en un ListView

04

HTTP

Consumir un JSON remoto

Contenido

1. Qué es JSON y por qué Flutter lo usa tanto
2. El JSON de este proyecto: enfermedades-laborales-cie11.json
3. Preparar el proyecto Flutter
4. Declarar el JSON como asset
5. Crear el modelo en Dart
6. Parsear el JSON con dart:convert
7. Mostrar los datos en la interfaz
8. Consumir un JSON desde internet (HTTP)
9. Errores frecuentes y cómo evitarlos
10. Buenas prácticas y siguiente paso

Cómo usar esta guía

Puedes copiar los fragmentos de código tal cual. Están pensados para el archivo `enfermedades-laborales-cie11.json`. Si tu JSON cambia de forma, solo ajustas el modelo.

1. Qué es JSON y por qué Flutter lo usa tanto

JSON (JavaScript Object Notation) es un formato de texto para intercambiar datos. En Flutter casi todo lo que llega de una API, un archivo local o un backend se representa como JSON: objetos con llaves y valores, listas, números, textos y booleanos.

Dart no “entiende” JSON por magia. Recibe un **String** (texto) y tú lo conviertes a estructuras nativas: `Map<String, dynamic>` y `List<dynamic>`. Después, lo ideal es mapear esas estructuras a **clases** con tipos claros (modelos). Así el resto de la app no trabaja con mapas sueltos, sino con objetos como `Enfermedad`.

Flujo mental (siempre el mismo)

Paso	Qué ocurre	En Dart
1. Origen	Lees un archivo o llamas a una URL	<code>rootBundle / http.get</code>
2. Texto	Obtienes un String con el JSON crudo	<code>String jsonStr</code>
3. Decode	Conviertes el texto a Map o List	<code>jsonDecode(jsonStr)</code>
4. Modelo	Pasas el Map a una clase con <code>fromJson</code>	<code>Enfermedad.fromJson(map)</code>
5. UI	Pintas listas, tarjetas, filtros, etc.	<code>ListView.builder</code>

2. El JSON de este proyecto

El archivo enfermedades-laborales-cie11.json no es una lista suelta. Es un **objeto raíz** con dos claves: meta (datos del catálogo) y enfermedades (array de 55 ítems).

Cada enfermedad mezcla tipos: id es número, agentes y sectores son listas de texto, y el resto son String. Eso define las propiedades de tu clase Dart.

Fragmento real del archivo

```
{
  "meta": {
    "titulo": "Enfermedades del CIE-11 asociadas a enfermedades laborales",
    "version_cie": "CIE-11 MMS",
    "total": 55
  },
  "enfermedades": [
    {
      "id": 1,
      "codigo_cie11": "CA60.0",
      "nombre": "Neumoconiosis por polvo de sílice (silicosis)",
      "categoria": "Respiratoria",
      "agentes": ["sílice cristalina"],
      "sectores": ["minería", "construcción", "canteras", "fundición"],
      "relacion_laboral": "Inhalación crónica de polvo de sílice en el trabajo"
    }
  ]
}
```

Observa los tipos: id es int, agentes y sectores son List<String>, el resto es String. Eso define exactamente las propiedades de tu clase Dart.

Campo JSON	Tipo Dart	Significado
id	int	Identificador interno del ítem
codigo_cie11	String	Código CIE-11 (ej. CA60.0)
nombre	String	Nombre clínico o oficial
categoria	String	Grupo clínico (Respiratoria, etc.)
agentes	List<String>	Exposiciones laborales típicas
sectores	List<String>	Sectores donde es frecuente
relacion_laboral	String	Por qué se asocia a enfermedad laboral

3. Preparar el proyecto Flutter

Si aún no tienes app, crea una con el CLI. Luego copia el JSON a una carpeta de assets. La estructura recomendada es esta:

```
flutter create cie11_app
cd cie11_app

# Copia el JSON al proyecto
mkdir -p assets/data
cp enfermedades-laborales-cie11.json assets/data/
```

Estructura de carpetas resultante (simplificada).

```
cie11_app/
  assets/
    data/
      enfermedades-laborales-cie11.json
  lib/
    models/
      enfermedad.dart
    services/
      cie11_service.dart
    screens/
      enfermedades_page.dart
    main.dart
  pubspec.yaml
```

4. Declarar el JSON como asset

Flutter no incluye archivos sueltos por defecto. Debes registrarlos en `pubspec.yaml`. Si omites este paso, `rootBundle.loadString` fallará con un error de asset no encontrado.

```
# pubspec.yaml
flutter:
  uses-material-design: true
  assets:
    - assets/data/enfermedades-laborales-cie11.json
    # o toda la carpeta:
    # - assets/data/
```

Importante

La indentación de YAML es estricta. `assets:` debe ir dentro de `flutter:`, con dos espacios. Después ejecuta `flutter pub get` y, si el archivo no aparece, detén la app y vuelve a lanzarla (un hot reload no siempre recarga assets nuevos).

5. Crear el modelo en Dart

El modelo es el puente entre el JSON y tu UI. Cada campo del JSON se convierte en una propiedad tipada. El `factory fromJson` es el corazón: recibe un `Map<String, dynamic>` (lo que devuelve `jsonDecode`) y construye el objeto.

lib/models/enfermedad.dart

```
class Enfermedad {
  final int id;
  final String codigoCie11;
  final String nombre;
  final String categoria;
  final List<String> agentes;
  final List<String> sectores;
  final String relacionLaboral;

  const Enfermedad({
    required this.id,
    required this.codigoCie11,
    required this.nombre,
    required this.categoria,
    required this.agentes,
    required this.sectores,
    required this.relacionLaboral,
  });

  factory Enfermedad.fromJson(Map<String, dynamic> json) {
    return Enfermedad(
      id: json['id'] as int,
      codigoCie11: json['codigo_cie11'] as String,
      nombre: json['nombre'] as String,
      categoria: json['categoria'] as String,
      agentes: List<String>.from(json['agentes'] as List),
      sectores: List<String>.from(json['sectores'] as List),
      relacionLaboral: json['relacion_laboral'] as String,
    );
  }
}
```

El catálogo (objeto raíz del JSON)

```
class CatalogoCie {
  final String titulo;
  final String versionCie;
  final int total;
  final List<Enfermedad> enfermedades;

  const CatalogoCie({
    required this.titulo,
    required this.versionCie,
    required this.total,
    required this.enfermedades,
  });

  factory CatalogoCie.fromJson(Map<String, dynamic> json) {
    final meta = json['meta'] as Map<String, dynamic>;
    final lista = json['enfermedades'] as List<dynamic>;
    return CatalogoCie(
      titulo: meta['titulo'] as String,
      versionCie: meta['version_cie'] as String,
      total: meta['total'] as int,
      enfermedades: lista.map((item) {
        return Enfermedad.fromJson(
          item as Map<String, dynamic>,
        );
      }).toList(),
    );
  }
}
```

Nota de estilo: en JSON los nombres van en snake_case (codigo_cie11). En Dart se usa camelCase (codigoCie11). El fromJson es el único lugar donde se mezclan ambos mundos.

Por qué List<String>.from y no un cast directo

jsonDecode tipa las listas como List<dynamic>. Un cast as List<String> suele fallar en runtime. List<String>.from(...) recorre cada elemento y lo convierte de forma segura.

6. Parsear el JSON con dart:convert

La biblioteca dart:convert viene con el SDK. No necesitas un paquete extra para decode básico. El servicio concentra la lectura del asset y el parseo, para que la pantalla solo pida “dame el catálogo”.

lib/services/cie11_service.dart

```
import 'dart:convert';
import 'package:flutter/services.dart';
import '../models/enfermedad.dart';

class Cie11Service {
  static const assetPath =
    'assets/data/enfermedades-laborales-cie11.json';

  Future<CatalogoCie> cargarCatalogo() async {
    final String texto = await rootBundle.loadString(assetPath);
    final Map<String, dynamic> mapa =
      jsonDecode(texto) as Map<String, dynamic>;
    return CatalogoCie.fromJson(mapa);
  }

  Future<List<Enfermedad>> cargarEnfermedades() async {
    final catalogo = await cargarCatalogo();
    return catalogo.enfermedades;
  }
}
```

Qué hace cada línea clave

- **rootBundle.loadString** lee el asset empaquetado y devuelve el JSON como texto.
- **jsonDecode** transforma ese texto en un Map anidado (objetos) y List (arrays).
- El **as Map<String, dynamic>** afirma que la raíz es un objeto, no un array.
- **CatalogoCie.fromJson** recorre *enfermedades* y crea cada modelo.

7. Mostrar los datos en la interfaz

Como la lectura es asíncrona, usa **FutureBuilder** o un **StatefulWidget** que cargue en **initState**. Aquí va la versión con **FutureBuilder**, compacta y clara para una guía.

lib/screens/enfermedades_page.dart

```

import 'package:flutter/material.dart';
import '../models/enfermedad.dart';
import '../services/cie11_service.dart';

class EnfermedadesPage extends StatelessWidget {
  const EnfermedadesPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('CIE-11 laborales')),
      body: FutureBuilder<CatalogoCie>(
        future: Cie11Service().cargarCatalogo(),
        builder: (context, snapshot) {
          if (snapshot.connectionState ==
              ConnectionState.waiting) {
            return const Center(
              child: CircularProgressIndicator(),
            );
          }
          if (snapshot.hasError) {
            return Center(
              child: Text('Error: ${snapshot.error}'),
            );
          }
          final catalogo = snapshot.data!;
          return ListView.builder(
            itemCount: catalogo.enfermedades.length,
            itemBuilder: (context, index) {
              final e = catalogo.enfermedades[index];
              return Card(
                child: ListTile(
                  leading: CircleAvatar(
                    child: Text('${e.id}'),
                  ),
                  title: Text(e.nombre),
                  subtitle: Text(
                    '${e.codigoCie11} · ${e.categoria}\n'
                    'Agentes: ${e.agentes.join(', ')}',
                  ),
                  isThreeLine: true,
                ),
              );
            },
          );
        },
      ),
    );
  }
}

```

lib/main.dart

```
import 'package:flutter/material.dart';
import 'screens/enfermedades_page.dart';

void main() => runApp(const Cie11App());

class Cie11App extends StatelessWidget {
  const Cie11App({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'CIE-11 laborales',
      theme: ThemeData(
        colorSchemeSeed: const Color(0xFF0E7C7B),
        useMaterial3: true,
      ),
      home: const EnfermedadesPage(),
    );
  }
}
```

Al ejecutar `flutter run` deberías ver las 55 enfermedades en una lista, cada una con código CIE-11, categoría y agentes.

8. Consumir un JSON desde internet (HTTP)

El parseo no cambia: sigue siendo `jsonDecode` + `fromJson`. Lo único distinto es el origen del String. En vez de `rootBundle` usas el paquete `http` (o `dio` / `dio_client`). Esto aplica cuando el JSON vive en un servidor, GitHub raw, Firebase Hosting, etc.

Dependencia

```
# pubspec.yaml
dependencies:
  flutter:
    sdk: flutter
  http: ^1.2.2
```

Servicio HTTP (mismo modelo)

```
import 'dart:convert';
import 'package:http/http.dart' as http;
import '../models/enfermedad.dart';

class Cie11HttpService {
  Future<CatalogoCie> cargarDesdeUrl(String url) async {
    final uri = Uri.parse(url);
    final respuesta = await http.get(uri);

    if (respuesta.statusCode != 200) {
      throw Exception(
        'HTTP ${respuesta.statusCode}: no se pudo cargar el JSON',
      );
    }

    final mapa = jsonDecode(respuesta.body) as Map<String, dynamic>;
    return CatalogoCie.fromJson(mapa);
  }
}
```

Android, iOS y CORS

En Android 9+ el tráfico HTTP claro (no HTTPS) está bloqueado salvo que lo habilites. En iOS, App Transport Security exige HTTPS. Si cargas desde un dominio web hacia Flutter web, el servidor debe enviar cabeceras CORS. Para un archivo local de clase, assets es más simple y no tiene esos problemas.

Permisos de red (Android)

El permiso de internet suele venir en la plantilla de Flutter. Confirma en `android/app/src/main/AndroidManifest.xml`:

```
<uses-permission android:name="android.permission.INTERNET"/>
```

9. Errores frecuentes y cómo evitarlos

Síntoma	Causa típica	Solución
Unable to load asset	El JSON no está en <code>pubspec.yaml</code> o la ruta no coincide	Declara el asset, revisa mayúsculas y reinicia la app
type 'List<dynamic>' is not a subtype of type 'List<String>'	Cast directo sobre un array JSON	Usa <code>List<String>.from(json['agentes'] as List)</code>
type 'Null' is not a subtype of type 'String'	Un campo falta o se llama distinto (typo)	Compara claves del JSON con <code>fromJson</code> . Usa <code>?</code> y valores por defecto si es opcional
type '_Map<String, dynamic>' is not a subtype of type 'List'	Tratas la raíz como lista, pero es un objeto { meta, enfermedades }	Decode a Map y lee <code>json['enfermedades']</code>
FormatException: Unexpected character	El archivo no es JSON válido (coma extra, comentarios)	Valida el JSON en un linter o <code>jsonlint</code>
XMLHttpRequest error (Flutter web)	CORS al pedir un JSON de otro origen	Sirve el archivo desde el mismo origen o usa assets

Campos opcionales (defensa ante JSON incompleto)

Si un campo puede faltar, no lo marques como required sin red de seguridad. Este patrón evita crashes cuando el backend omite una clave:

```
factory Enfermedad.fromJson(Map<String, dynamic> json) {
  return Enfermedad(
    id: json['id'] as int? ?? 0,
    codigoCie11: json['codigo_cie11'] as String? ?? '',
    nombre: json['nombre'] as String? ?? 'Sin nombre',
    categoria: json['categoria'] as String? ?? 'Sin categoría',
    agentes: List<String>.from(json['agentes'] ?? const []),
    sectores: List<String>.from(json['sectores'] ?? const []),
    relacionLaboral: json['relacion_laboral'] as String? ?? '',
  );
}
```

10. Buenas prácticas y siguiente paso

- **Separa capas.** Modelo, servicio y UI en archivos distintos. Mañana puedes cambiar assets por HTTP sin tocar ListView.
- **Un fromJson por clase.** No parsees Map sueltos en el widget.
- **Maneja loading y error.** El usuario debe ver un indicador o un mensaje, nunca una pantalla en blanco.
- **No bloquee el hilo.** loadString y http.get son async. Nunca uses operaciones síncronas pesadas en build().
- **Filtra en memoria.** Con 55 ítems puedes filtrar por categoría o buscar por nombre con `.where((e) => e.categoria == 'Respiratoria')`.
- **Cuando el JSON crezca,** considera json_serializable o freezed para generar fromJson y evitar errores de tipeo.

Mini ejemplo: filtrar por categoría

```
List<Enfermedad> filtrarPorCategoria(
  List<Enfermedad> todas,
  String categoria,
) {
  return todas
    .where((e) => e.categoria.toLowerCase() == categoria.toLowerCase())
    .toList();
}

List<Enfermedad> buscar(List<Enfermedad> todas, String q) {
  final query = q.toLowerCase();
  return todas.where((e) {
    return e.nombre.toLowerCase().contains(query) ||
           e.codigoCie11.toLowerCase().contains(query) ||
           e.agentes.any((a) => a.toLowerCase().contains(query));
  }).toList();
}
```

Checklist final

#	Verificación
1	El JSON está en assets/data y declarado en pubspec.yaml
2	Enfermedad.fromJson cubre los 7 campos de cada ítem
3	CatalogoCie.fromJson lee meta y recorre enfermedades
4	El servicio usa jsonDecode y no parsea a mano con substring
5	La UI muestra waiting, error y lista (los tres estados)
6	agentes y sectores se construyen con List<String>.from
7	Si usas HTTP, compruebas statusCode == 200 antes de decode

Resumen en una frase

Lees un String (asset o HTTP), lo conviertes con jsonDecode a Map, mapeas ese Map a clases Dart con fromJson, y pintas la lista. Ese ciclo es el mismo para este CIE-11 y para cualquier API REST.

Documento de apoyo académico. Los códigos CIE-11 del JSON de ejemplo son de referencia; verificar en el navegador oficial de la OMS (icd.who.int) antes de uso normativo o clínico.